Pamantasan ng Cabuyao (Cabuyao University)
Bachelor of Science in Computer Science

Centralized Student Portal and Automated Clearance System

System Requirements & Project Documentation

Member	Role
Abarquez, Justinbril	Project Manager / Systems Analyst
Cruzante, Aaron	Back-End Developer
Canayong, Ron	Back-End Developer
Esgana, Justine	Front-End Developer
Espedilon, Arvien	Front-End Developer
Dominado, Byrone	Front-End Developer

Tech stack: Next.js, Tailwind CSS, Supabase (PostgreSQL), Vercel

Project duration: 12 days (Agile, short sprints)

Prototype: https://cabuyaouni.vercel.app

Source code: https://github.com/brtbrt123/sv-student-portal

Date: October 2026

Table of Contents

1.	SDLC Plan	3
2.	System Modeling Diagrams (DFD, Use Case, Activity, Class)	6
3.	UI/UX Screen Designs & Wireframes	9
4.	Database Design (ERD / Normalization)	10
5.	Software Testing Plan	12
6.	Ethical Analysis	15
7.	Prototype Link & Source Code	17
8.	References	19
9.	Verification Reports	20
10.	Team Monitoring Sheet	21

1. SDLC Plan

1.1 Project overview

This project delivers a comprehensive, web-based university portal designed to eliminate physical queuing for enrollment, automate the student clearance process, and centralize grade viewing and document requests. Due to the accelerated 12-day deadline, the system uses a decoupled Backend-as-a-Service (BaaS) architecture to ensure rapid deployment, scalability, and security without sacrificing architectural integrity.

1.2 Objectives

- **Digitize Enrollment:** Allow students to submit their enrollment (subject and section selection) online and allow the Registrar to review and approve it, removing the need for physical queuing.
- **Automate Clearances:** Reduce physical clearance processing time by digitizing approvals across academic and financial departments.
- **Centralize Academic Data:** Provide a secure portal for students to view grades and academic standing in real time.
- **Streamline Requests:** Implement a ticketing system for document requests (e.g., Transcript of Records, Certificates) to eliminate physical line queuing.
- **Modernize Financial Tracking:** Enable the Cashier module to digitally tag payments and automatically clear the Cashier's Office clearance item once payment is recorded.

1.3 Scope

In-Scope:

- Development of 6 core modules: Student Portal, Registrar, Cashier, Department, Admin, and Document Request System, plus three features added in Sprint 4: the public application and enrollment site (Admissions), the virtual queue for the Registrar window, and in-app notifications.
- Online enrollment submission (Student Portal) and enrollment approval (Registrar), as covered in the SRS.
- Database design normalized to the Third Normal Form (3NF).
- Role-Based Access Control (RBAC) ensuring strict data isolation.
- Implementation of system security protocols: Rate Limiting, Row Level Security, Brute-Force Protection, and SQL Injection prevention (parameterized queries).
- Live cloud deployment via Vercel.

Out-of-Scope:

- Integration with external payment gateways (credit cards, e-wallets); cashier transactions will be manually tagged by staff for this phase.
- Hardware procurement or physical kiosk setup.

1.4 Team and roles

- **Project Manager / Systems Analyst (1):** Acts as Product Owner and Scrum Master (manages the backlog, sprint plan and timeline), designs the system models (DFD, Use Case, Activity, and Class Diagrams), writes the SDLC plan, test plan, and ethical analysis, compiles the final IEEE documentation in parallel with development, and is the only person who merges dev into main for deployment.
- **Frontend Developers (3):** Develop UI/UX wireframes and storyboards and build responsive Next.js/Tailwind CSS components that read live data from the database.
- **Backend / Database Developers (2):** Own the ERD and 3NF PostgreSQL schemas, configure Supabase Authentication, implement Row Level Security (RLS) policies, build login lockout and rate limiting,

implement the audit log, and connect live APIs.

- *Ownership note:* The PM reviews the ERD for consistency with the Class Diagram, but the Backend team is the owner of the ERD and 3NF tables so the work is not duplicated.

1.5 Selected model: Agile (Scrum-style)

The project has a fixed final deadline (Oct 15, 2026) and a short window, and several parts depend on each other (authentication, then clearance, then the modules). Short sprints let the team show working software from the `dev` branch at the end of every phase, find integration problems early, and re-prioritize when a sprint runs late. A plan-driven (waterfall or RAD) approach would find those problems only at the end. The process follows the Scrum Guide [6].

1.6 SDLC phases

Phase	Activities	Output
Planning and requirements	Project Charter, SRS, requirement IDs (REQ, SEC, PER, ETH)	This document
Design	DFD, Use Case diagram, ERD/3NF, UI/UX wireframes in Figma	`docs/diagrams` (SVG, regenerated by `scripts/diagrams/`), Figma file `Cabuyao_Uni` (screens), Figma file for the six diagrams (https://www.figma.com/design/caIH5zcYI3ursQ9c2iYFJg)
Implementation	Next.js 16 (App Router, TypeScript), Tailwind CSS 4, Supabase (PostgreSQL, Auth, Row Level Security)	`main` and `dev` branches
Testing	Role-routing and data-isolation tests, security tests (SQL injection, brute force, rate limiting, RLS), integration testing	`docs/testing-plan.md`
Deployment	Vercel deploys every change to `main`	Live prototype
Documentation and review	IEEE paper, plagiarism and AI-detector reports, Team Monitoring Sheet	Final deliverables

1.7 Sprints

Sprint	Goal	Outcome
Sprint 1	System modeling, ERD/3NF design, UI/UX wireframes, database initialization	Diagrams, schema and wireframes
Sprint 2	Authentication and role routing, ERD v3 migration, shared UI and dashboard shell, SQL files, SDLC plan	Role-based login and the app shell
Sprint 3	Clearance logic first (holds, department approval, Cashier auto-clear, final approval as database triggers), then the Registrar, Cashier and Department modules and the Admin audit log	The clearance chain on live data
Sprint 4	Enrollment, grades, document requests, virtual queue, notifications, applications; Admin screens and dashboards; login lockout, rate limiting and security hardening; testing plan; release	Complete system on the live site

1.8 Team roles in the process

Role	Members	Responsibility
Product Owner and Scrum Master	Abarquez, Justinbril	Backlog order, sprint reviews, merges `dev` into `main`

Role	Members	Responsibility
Backend team	Cruzante, Aaron; Canayong, Ron	Database, RLS, authentication, triggers, security
Frontend team	Esgana, Justine; Espedilon, Arvien; Dominado, Byrone	UI from the Figma design, module screens

1.9 Backlog by dependency

1. Authentication and role-based access (SEC-01, SEC-02, REQ-ADM-02)
2. Shared UI, navigation and dashboard shell
3. Clearance chain: holds, department approval, Cashier auto-clear, final approval (REQ-CSH-03, REQ-REG-03) and audit logging (REQ-ADM-03)
4. Module screens: Student, Registrar, Cashier, Department, Admin (REQ-STU, REQ-REG, REQ-CSH, REQ-DPT, REQ-ADM, REQ-DOC)
5. Security hardening: lockout, rate limiting, injection checks (SEC-03 to SEC-05)
6. Deployment and final documents

1.10 Branching and deployment flow

1. Nobody codes directly on `main` or `dev`.
2. Each task gets a branch from `dev` named `feature/<task>` (or `chore/<task>`, `docs/<task>`).
3. The author opens a pull request into `dev`.
4. When `dev` is tested, the Project Manager merges `dev` into `main`.
5. Vercel deploys every change to `main` automatically. Secret keys stay in server-only code (SEC-06).

1.11 Risks and mitigation

Risk	Mitigation
A sprint runs late and squeezes the next one	Order the backlog by dependency; re-prioritize at each sprint review
A pull request is merged into the wrong branch	Check the base branch of every pull request; only the Project Manager merges into `main`
Rules between tables are not enforced (clearance chain)	Implement them as database triggers early in Sprint 3 and test each one
Keys leak into the repository	`.env.local` is git-ignored; `service_role` never reaches the browser
Design and code drift apart	Build screens from the Figma design using the shared design tokens

2. System Modeling Diagrams (DFD, Use Case, Activity, Class)

The system is modeled with a Data Flow Diagram, a Use Case Diagram, an Activity Diagram and a Class Diagram, all drawn from the real requirements and database design. They are kept in one Figma file with editable layers: https://www.figma.com/design/caIH5zcYI3ursQ9c2iYFJg

Diagram	What it shows	Link
Data Flow Diagram, level 1	13 processes, 14 data stores, 10 sections	https://www.figma.com/design/caIH5zcYI3ursQ9c2iYFJg?node-id=10-2
Use Case Diagram	6 actors and their use cases	https://www.figma.com/design/caIH5zcYI3ursQ9c2iYFJg?node-id=8-2
Activity Diagram (term clearance)	5 swimlanes: Student, Department, Cashier, Registrar, System	https://www.figma.com/design/caIH5zcYI3ursQ9c2iYFJg?node-id=10-522
Class Diagram	28 classes with attributes, operations, generalization and multiplicities	https://www.figma.com/design/caIH5zcYI3ursQ9c2iYFJg?node-id=12-2

2.1 Functional requirements (basis of the models)

Module	Requirements
Student	REQ-STU-01 sign in with student ID or email; REQ-STU-02 see enrollment status and grades; REQ-STU-03 see clearance across all departments; REQ-STU-04 request documents and track them; REQ-STU-05 submit an online enrollment; REQ-STU-06 request the term clearance
Registrar	REQ-REG-01 encode and update grades; REQ-REG-02 process document requests; REQ-REG-03 final clearance only when every item is cleared; REQ-REG-04 review, approve or reject enrollments
Cashier	REQ-CSH-01 view balances; REQ-CSH-02 log payments, tag charges Cleared or Waived; REQ-CSH-03 Finance clearance updates automatically
Department	REQ-DPT-01 see only own students; REQ-DPT-02 place and lift holds; REQ-DPT-03 approve departmental clearance when holds are lifted
Admin	REQ-ADM-01 manage user accounts; REQ-ADM-02 role-based access control; REQ-ADM-03 read-only audit log; REQ-ADM-04 maintain the academic catalog
Document requests	REQ-DOC-01 unique ticket per request (one to five documents); REQ-DOC-02 status updates when the Registrar processes it
Admissions	REQ-APP-01 public application form; REQ-APP-02 Registrar accepts or rejects and the account is created; REQ-APP-03 automatic first-year enrollment for new students
Virtual queue	REQ-QUE-01 rotating QR queue pass; REQ-QUE-02 Registrar scans, calls next, Done or No-show; REQ-QUE-03 place in line, hold and cancel; REQ-QUE-04 every ticket change is role-checked and audited
Notifications	REQ-NOT-01 in-app inbox for every role
Security and performance	SEC-01 to SEC-08 (authentication, RLS, lockout, rate limiting, SQL injection, secrets, password rules, headers); PER-01 pages under 2.5 s; PER-02 queries under 500 ms; PER-03 continuous deployment

Requirements follow the IDs and style of ISO/IEC/IEEE 29148 [7].

2.2 Data Flow Diagram (level 1)

The diagram has 13 processes and 14 data stores, arranged in 10 sections (A to J). Each section shows one area, and the same entity or store can appear in several sections. Each arrow is labelled with the data it carries. Data stores are reached only through processes. Dashed arrows are automatic changes made by

database triggers. Student payments are not an input: the Cashier logs payments, because no payment gateway is integrated (see the charter).

Section	Processes	Requirements
A. Authentication	1.0 Authenticate users	REQ-STU-01, SEC-03, SEC-07
B. Admissions	2.0 Process admissions	REQ-APP-01 to REQ-APP-03
C. Enrollment	3.0 Manage enrollment	REQ-STU-02, REQ-STU-05, REQ-REG-04
D. Grades	4.0 Manage grades	REQ-STU-02, REQ-REG-01
E. Payments and finance clearance	5.0 Process payments	REQ-CSH-01 to REQ-CSH-03
F. Clearance	6.0 Initiate and track clearance, 7.0 Clear departments and manage holds, 8.0 Issue final clearance	REQ-STU-03, REQ-STU-06, REQ-DPT-01 to REQ-DPT-03, REQ-REG-03
G. Document requests	9.0 Process document requests	REQ-STU-04, REQ-REG-02, REQ-DOC-01, REQ-DOC-02
H. Virtual queue	10.0 Manage the queue	REQ-QUE-01 to REQ-QUE-04
I. Notifications	11.0 Send and show notifications	REQ-NOT-01
J. Administration and audit trail	12.0 Manage accounts, roles and catalog, 13.0 Record audit trail	REQ-ADM-01 to REQ-ADM-04

The data stores group the tables of the ERD:

Store	Tables
D1 Accounts	`roles`, `profiles`, `students`, `department_staff`
D2 Login attempts	`login_attempts`
D3 Academic catalog	`departments`, `programs`, `terms`, `subjects`, `sections`
D4 Enrollments	`enrollments`, `enrollment_items`
D5 Grades	`grades`, `grade_scale`
D6 Finance	`charges`, `payments`, `fee_types`
D7 Clearances	`clearances`, `clearance_items`
D8 Academic holds	`academic_holds`
D9 Document requests	`document_requests`, `document_types`
D10 Audit log	`audit_logs`
D11 Applications	`applications`
D12 Program curriculum	`program_curriculum`
D13 Queue	`queue_services`, `queue_tickets`, `queue_counters`
D14 Notifications	`notifications`

External entities are the Applicant (no account), the Student, Registrar, Cashier, Department Head, and Admin, all users at login, and Supabase Auth as an external service. The acceptance email is sent to the Applicant by the email service (Resend).

2.3 Use Case Diagram

The diagram has 6 actors: Applicant (not signed in), Student, Registrar, Cashier, Department Head, and Admin. The five signed-in actors are connected to "Log in (Supabase Auth)" (orange line) and to "View Notifications" (blue line), and every use case except "Apply for Admission" requires a signed-in user. "Set Password" extends

"Log in" for a first sign-in. A solid line is an association, and a dashed arrow is an «include» (base to included) or an «extend» (extension to base).

Actor	Use cases
Applicant	Apply for Admission (New student or Transferee)
Admin	Create, Update and Deactivate Accounts; Assign Roles (RBAC); Maintain Academic Catalog; View Audit Logs
Department Head	View Students Pending Clearance; Place or Lift Academic Holds; Approve Departmental Clearance («include» Check Active Holds)
Cashier	View Financial Balances; Log Payments and Tag Charges Cleared («include» Update Finance Clearance, automatic, REQ-CSH-03)
Registrar	Review Applications (Accept or Reject; «include» Create Student Account, which sends the acceptance email); Drop a Subject from a First-Year Enrollment; Run the Queue Window (call, hold, no-show, done; «include» Scan Student QR Code); Review Enrollments (Approve or Reject); Encode and Update Grades; Process Document Requests (extended by Reject Document Request when the request cannot be fulfilled); Issue Final Clearance («include» Verify All Departments and Finance Cleared)
Student	View Enrollment Status; Submit Online Enrollment; View Academic Grades; Request Clearance; Check Clearance Status; Submit Document Request (one to five documents); Track Document Request; Get Queue Ticket (QR code); Cancel Queue Ticket; View Notifications

2.4 Activity Diagram (term clearance)

The diagram shows the clearance process in five swimlanes (Student, Department, Cashier, Registrar, System). The Student requests the term clearance (REQ-STU-06) and the system creates one clearance item per department, including Finance. The Department and Cashier work in parallel: a department approves its item once all of its holds are lifted (REQ-DPT-03), and the Finance item is cleared automatically when every charge is Cleared or Waived (REQ-CSH-03, database trigger). When all items are cleared, the Registrar issues the final clearance approval (REQ-REG-03), the system sets the clearance to Approved and writes the audit log (REQ-ADM-03), and the Student sees the result.

2.5 Class Diagram

The diagram is a UML class diagram of the portal. It has 28 classes, each with its main attributes (`-`) and its operations (`+`). The abstract class `User` is specialised by `Student`, `Registrar`, `Cashier`, `DepartmentHead` and `Admin` (hollow triangle, generalization), and each role class lists what that role can do in the system (for example `Registrar.reviewApplication()`, `Student.getQueueTicket()`, `Cashier.logPayment()`). The domain classes are `Application`, `Enrollment`, `EnrollmentItem`, `Grade`, `GradeScale`, `Clearance`, `ClearanceItem`, `AcademicHold`, `Charge`, `Payment`, `DocumentRequest`, `QueueTicket`, `QueueService`, `Notification`, `AuditLog`, `Program`, `ProgramCurriculum`, `Department`, `Subject`, `Section` and `Term`. Lines are associations with multiplicities (`1`, `*`, `0..1`, `1..*`); a filled diamond marks composition (an enrollment owns its items, a clearance owns its items). Links that every class shares, such as the term of a charge or the actor who made a change, are left out to keep the diagram readable. The classes are colored by module like the ERD, and the tables behind them are in the ERD (Figures 1 and 2).

3. UI/UX Screen Designs & Wireframes

Mid-fidelity wireframes of every screen, grouped by role, are in one Figma file: https://www.figma.com/design/ZtlUFDtPCuTFA1V2SCIlrb

Group	Screens
Public (5)	Landing, Login (Student and Staff tabs), Apply for admission, Application received, Set password
Student (7)	Dashboard, Enrollment with queue pass, Grades, Clearance, Document requests, Queue, Notifications
Registrar (7)	Dashboard, Applications, Enrollments, Encode grades, Queue desk, Final clearance, Document requests
Cashier (4)	Dashboard, Charges, Payments, Finance clearance
Department (3)	Dashboard, Clearance items, Academic holds
Admin (4)	Dashboard, Users, Academic setup, Audit log

3.1 Consistent UI/UX

Every signed-in role uses the same layout: a sidebar with the links of that role, a top bar with the notification count, and a content area made of cards and tables. The same shared components (buttons, inputs, cards, tables, status badges) appear on every screen, and colors come from design tokens (light and dark theme, one font family). The final screens follow the wireframes and are live at https://cabuyaouni.vercel.app.

3.2 UX principles

- **Clear status:** status is shown as a word badge (Pending, Cleared, Approved), never by color alone; a student sees every clearance office, who is holding it up, the reason, and the next step.
- **Safe actions:** a button that cannot be used is disabled and says why; destructive actions are red and ask for confirmation.
- **Plain messages:** database rules return fixed messages that the app turns into sentences, for example "Approval is blocked. Lift the active hold first."
- **Every state designed:** loading, empty and error states on each screen.
- **Phone and accessibility:** every page works at phone width (lists replace wide tables, touch targets of 44 px), keyboard focus is visible, and a dark theme is available.

3.3 Screen overview by role

Role	What the screens let the user do
Applicant	Apply for admission as a new student or a transferee; receive the account by email
Student	See clearance status per office, grades and GWA per term, enroll, request documents, get and hold a queue ticket, read notifications
Registrar	Accept applications, approve enrollments, encode grades, process documents, run the queue window, issue final clearance
Cashier	See balances, record payments, tag charges; the Finance clearance item updates by itself
Department head	See only own students, place and lift holds, approve the department's clearance item
Admin	Manage staff accounts and roles, maintain terms, programs, departments, subjects and sections, read the audit log

4. Database Design (ERD / Normalization)

Entity Relationship Diagrams: Overview https://www.figma.com/design/caIH5zcYI3ursQ9c2iYFJg?node-id=4-2 and Detailed, with every column, type and key, https://www.figma.com/design/caIH5zcYI3ursQ9c2iYFJg?node-id=5-2

The database is defined in `schema.sql` (the first 23 tables) and in the rule files `applications_rules.sql`, `first_year_curriculum.sql`, `queue_rules.sql` and `notifications_rules.sql` (the other 6; PostgreSQL 16 on Supabase), and documented in the database design file (data dictionary, normalization proof up to 3NF, and rules R1 to R12). It has 29 tables in 10 modules, one external table (Supabase `auth.users`), 47 foreign keys (including `profiles` to `auth.users`), and 4 views (`v_clearance_items`, `v_clearances`, `v_active_holds`, `v_grades`). The tables, by module:

- **Identity & Access:** `roles`, `profiles`, `students`, `department_staff`.
- **Academic Structure:** `departments`, `terms`, `programs`, `subjects`, `sections`, `program_curriculum`.
- **Enrollment & Grades:** `grade_scale`, `enrollments`, `enrollment_items`, `grades`.
- **Finance:** `fee_types`, `charges`, `payments`.
- **Document Requests:** `document_types`, `document_requests`.
- **Clearance:** `clearances`, `clearance_items`, `academic_holds`.
- **Admissions:** `applications`.
- **Queue:** `queue_services`, `queue_tickets`, `queue_counters`.
- **Notifications:** `notifications`.
- **Security & Audit:** `audit_logs`, `login_attempts`.

Main design points:

- **Accounts:** credentials stay in Supabase Auth. `profiles.profile_id` has the same value as `auth.users.id`, so no password is stored in a portal table (SEC-01). `students` and `department_staff` are subtype tables of `profiles`.
- **Finance as a department:** the Cashier's Office is a row in `departments` with type Financial, so every row of `clearance_items` points to a department in the same way.
- **No stored status when a timestamp already says it:** `clearances`, `clearance_items`, and `academic_holds` have no status column. Their status is read from the views `v_clearances`, `v_clearance_items`, and `v_active_holds`.
- **Grades:** `grades` stores only `grade_code`. The numeric value and pass/fail come from `grade_scale`.
- **Balances:** `payments` reference a charge (`charge_id`) and carry no `student_id`. A balance is computed from `charges` and `payments`. `charges.status` (Unpaid, Cleared, Waived) is stored because the Cashier tags it by hand.
- **Unique keys:** one enrollment per student per term, one grade per enrolled section, one clearance per student per term, one clearance item per department per clearance, and unique receipt numbers and ticket numbers.
- **Security:** Row Level Security is enabled on all 29 tables (SEC-02), `audit_logs` is insert-only (UPDATE and DELETE are revoked), and foreign keys use `ON DELETE RESTRICT`, so users are deactivated with `profiles.is_active`, never deleted.
- **Applications, the queue and notifications:** clients cannot write `applications`, `queue_*` or `notifications` directly. Applications are created by `submit_application()`, queue tickets by role-checked functions (`issue_queue_ticket`, `queue_call_next`, `queue_finish`, and others), and notifications by triggers; a trigger on each table checks the status flow.
- **Rules R1 to R12:** cross-table rules (role consistency, enrollment limits, clearance creation, final approval, audit triggers, lockout) are enforced by triggers, policies, or the API. They are listed in the database design file, Section 7, and are not part of `schema.sql`.

4.1 Normalization to the Third Normal Form

- **1NF:** every column holds one value; repeating groups are separate tables (`enrollment_items` for the subjects of an enrollment, `clearance_items` for the offices of a clearance, `queue_tickets` for the tickets of a service).
- **2NF:** tables with a composite key keep only attributes that depend on the whole key (`program_curriculum` is keyed by program, year, semester and subject; `queue_counters` by service and day).
- **3NF:** no attribute depends on another non-key attribute. A grade stores only the grade code, and its numeric value and pass flag are in `grade_scale`; a balance is computed from `charges` and `payments`; the status of a clearance, item or hold is derived from timestamps in views; a department's type is stored once in `departments`; a student's program is stored once in `students`.

The schema follows the relational model [5].

5. Software Testing Plan

The full test cases, steps, expected results and recorded results are in the repository: https://github.com/brtbrt123/sv-student-portal/blob/main/docs/testing-plan.md

5.1 Test levels

Level	What is tested and how	Tools and location
Unit	Pure functions: password rules, student ID and login input, queue QR token and line, notification text, CSV, catalog validation, schedule parsing	Vitest, UT-01 to UT-128 (https://github.com/brtbrt123/sv-student-portal/tree/main/tests/unit)
Integration	Database rules and the app-to-database path: triggers, grants, Row Level Security, lockout, grades, documents, enrollment, applications, queue, notifications, admin. Each change is rehearsed in a rolled-back transaction, run live, then the regression script is re-run	SQL suites TC-01 to TC-259 (https://github.com/brtbrt123/sv-student-portal/tree/main/docs/database)
System (end to end)	Browser walks of the real screens as each role: the clearance chain, grades, documents, enrollment, queue, first-year enrollment, Admin screens and dashboards; phone width, dark mode and accessibility; a check of the deployed site after each release	E2E-01 to E2E-13, S-01 to S-12 and the system tests of the full plan
Acceptance	One check per requirement on the live site (section 5.3)	ACC-01 to ACC-10
Performance	PER-01: the main page of each role loads in under 2.5 s. PER-02: grade and clearance queries under 500 ms	PERF-01, PERF-02
Security	Isolation, grants, authentication, lockout, rate limiting, headers, injection and cross-user tests	V-1 to V-7, local security checks, live header tests

5.2 Test case groups

Test cases	What is checked
TC-12 to TC-71	Clearance chain: a department cannot clear an item while it has an active hold; the Finance item clears by itself when every charge is Cleared or Waived; final clearance only when every item is cleared; a department cannot read or approve another department's items
TC-72 to TC-82	Login lockout: 5 failures lock the account with exponential backoff; the check runs before the password; anonymous users cannot call private functions
TC-83 to TC-122	Grades only for items of an Approved enrollment; document status flow Pending, Processing, Ready, Released; enrollment rules (one per student and term, free seat, final decisions)
TC-123 to TC-166	Applications (accept and reject, duplicate emails give no hint), student ID sign-in, the virtual queue (ticket rules, random codes, status flow)
TC-167 to TC-259	Notifications (a failing notification never blocks the action), queue hold and No-show, academic catalog, Admin user accounts, automatic first-year enrollment
UT-01 to UT-128	Unit tests of the pure functions listed above
E2E and S series	Browser steps per role with the expected screen text and the audit rows written
PERF-01, PERF-02	Page load per role and query time per view

5.3 Acceptance test script

ID	Requirement	Acceptance check
ACC-01	REQ-STU-01	A student signs in with the student ID and lands on the student home
ACC-02	REQ-STU-02, REQ-REG-01	A student sees their own grades; the Registrar encodes a grade
ACC-03	REQ-STU-03, REQ-STU-06	A student requests clearance and sees every office
ACC-04	REQ-STU-04, REQ-DOC-01, REQ-DOC-02	A student requests a document, gets a ticket, and the Registrar moves it to Released
ACC-05	REQ-STU-05, REQ-REG-04	A student submits an enrollment; the Registrar approves or rejects it
ACC-06	REQ-DPT-01 to REQ-DPT-03	A department lifts a hold and approves only its own students
ACC-07	REQ-CSH-01 to REQ-CSH-03	The Cashier logs a payment; Finance clears by itself
ACC-08	REQ-REG-03	The Registrar can issue the final clearance only when every office has cleared
ACC-09	REQ-ADM-02, REQ-ADM-03	A role cannot open another role's module; the Admin reads the audit log
ACC-10	REQ-APP-01 to REQ-APP-03, REQ-QUE-01 to REQ-QUE-03	An applicant applies, the Registrar accepts, the student gets a queue ticket

5.4 Performance results

Test	Result	Limit
PERF-02: grade and clearance queries (30 runs each, as a student, Row Level Security on)	0.09 to 1.46 ms per query	500 ms
PERF-01: full page load of the main page of each role on the live site	0.93 to 2.18 s	2.5 s

5.5 Security tests: vulnerabilities and solutions

ID	Vulnerability	Solution	Test evidence
V-1	A Supabase helper function was executable by the anonymous role	EXECUTE revoked from `anon`	TC-81 failed before the fix and passes after it
V-2	Brute-force on the login	Login lockout with exponential backoff (5 failures, 60 s doubling, cap 15 min), checked before the password	TC-72 to TC-82; 5 wrong passwords lock the account in the app
V-3	Request flooding and denial of service	Vercel firewall rule: 20 POST requests per minute per IP, then HTTP 429	Live test: 40 POSTs gave 20 allowed and 20 blocked
V-4	A rule conflict blocked document remarks while Pending	The database sets the fields itself	14 cases pass
V-5	SQL injection	Only parameterized Supabase calls; no SQL built from user input	Injection payloads in the login, purpose, remarks, hold reason, receipt number and the Data API were stored as plain text and no table was changed

ID	Vulnerability	Solution	Test evidence
V-6	Queue QR forgery, replay and sharing	Signed token (HMAC-SHA256, server secret) that changes every 15 seconds	Unit tests UT-50 to UT-66
V-7	Proxy queuing	Random 7-digit ticket codes; the Registrar checks the student ID at the window	Queue checks Q-01 to Q-09

Other security results: 24 of 24 local repository checks; 0 known vulnerabilities in production dependencies; the 7 security headers present on the live site; the anonymous Data API refused on 17 of 17 probes; reads between two student accounts refused on 9 of 9 probes; other roles' pages redirect for a signed-in Student, Cashier and Registrar. The security design follows common web guidance [10].

6. Ethical Analysis

The system holds student records: grades, enrollment, balances, holds, document requests and clearance. A wrong or leaked record can hurt a real person, so each theme asks what the principle requires, what the system does, and what we propose. Principles come from the ACM and IEEE codes of ethics [1], [2] and the Data Privacy Act of 2012 [8].

6.1 Stakeholders and possible harm

Stakeholder	Need	Possible harm
Students	Correct, private records; a fair clearance; clear reasons for holds	Another person reads their grades or balance; a wrong hold or grade delays clearance
Registrar, Cashier, Department staff	Tools to do their job	Blamed for changes they did not make if the log is weak
Admin	Manage accounts, catalog and the audit log	Holds much power, so their own actions must be visible
The university	Trustworthy records and legal compliance	A data breach, loss of trust, complaints under the Data Privacy Act

6.2 Codes of ethics applied

The ACM Code of Ethics and Professional Conduct (2018) and the IEEE Code of Ethics are applied by theme: avoid harm, respect privacy and confidentiality, be honest about what was built, be accountable, and give proper credit.

6.3 Data Privacy Act of 2012 (RA 10173): principles and how the system meets them

Principle	How the system meets it
Transparency	Students see their own records, status and the reason for a hold
Legitimate purpose	Data is used only for enrollment, grades, documents and clearance; no analytics and no third-party sharing
Proportionality	A minimal schema, column-level grants, no hard deletes, fictional seed data
Confidentiality	Row Level Security on all 29 tables; each student reads only their own rows; each department sees only its own items
Integrity	Database triggers enforce the rules for every role; the audit log is append-only and nobody, including the Admin, can edit it
Availability	Hosted on Vercel and Supabase; performance measured against PER-01 and PER-02
Access control	Supabase Auth, login lockout, rate limiting, password rules, security headers
Accountability	Every critical change is written to the audit log by database triggers

6.4 Fairness and accessibility

The same rules apply to every role because the database enforces them, so there is no silent exception. Every screen works at phone width, has visible keyboard focus and a dark theme, and shows status in words and not by color alone.

6.5 Use of AI assistance

An AI coding assistant helped draft code, SQL, tests and documents. The team reviewed and tested every output (type check, lint, build, rolled-back SQL rehearsals and browser tests), approved every change to the

live database, never gave the assistant any secret key, and commits made with it carry a co-author line.

6.6 Proposed solutions and safeguards

Risk	Proposed solution
A student does not know what data is kept	A short privacy notice for students and a notice about the audit log for staff
A data breach	A written incident procedure that names who notifies the National Privacy Commission
Sensitive reads are not logged	Read logging for sensitive tables
The Cashier can edit amounts	Keep every edit audited and add a second approver for amount changes
Old login attempts and notifications pile up	Automatic retention (delete after a set time)
A wrong hold or grade	A correction request flow for students
Real data later	A review by the university's data protection contact before any real data is loaded

7. Prototype Link & Source Code

Item	Link
Live deployment (working prototype)	https://cabuyaouni.vercel.app
GitHub repository (source code)	https://github.com/brtbrt123/sv-student-portal
Test cases (full list)	https://github.com/brtbrt123/sv-student-portal/blob/main/docs/testing-plan.md
System diagrams	https://www.figma.com/design/caIH5zcYI3ursQ9c2iYFJg
Wireframes	https://www.figma.com/design/ZtlUFDtPCuTFA1V2SCIlrb

7.1 About the prototype

Item	Detail
Architecture	Next.js 16 (App Router, TypeScript) and Tailwind CSS on Vercel; Supabase for PostgreSQL, Authentication and Row Level Security; functions run in the same region as the database [9]
Roles	Student, Registrar, Cashier, Department head, Admin, and a public application site for applicants
Features	Public application and enrollment site; automatic first-year enrollment; student enrollment, grades, clearance and document requests; virtual queue with a rotating QR pass, hold and No-show; in-app notifications; Registrar, Cashier, Department and Admin screens; academic setup and user accounts; audit log; login lockout, rate limiting, security headers and password rules
Deployment	Every merge into the `main` branch deploys automatically to Vercel

7.2 Test accounts

Students sign in on the Student tab with their 8-digit student ID; staff sign in on the Staff tab with their email. The accounts hold fictional data only.

Role	Sign-in
Admin	`admin@pnc.test`
Registrar	`registrar@pnc.test`
Cashier	`cashier@pnc.test`
Department head	`dept@pnc.test`
Students	20260001, 20260002, 20260003

Passwords are given to the evaluators by the Project Manager.

7.3 Quick walkthrough

1. Sign in as a student and request clearance: four offices appear.
2. Sign in as `cashier@pnc.test` and record a payment: the Finance office clears by itself.
3. Sign in as `dept@pnc.test`, lift a hold and approve the department.
4. Sign in as `registrar@pnc.test` and issue the final clearance.
5. Sign in as `admin@pnc.test` and open the audit log to see every step recorded.

7.4 Run the source code

```
git clone https://github.com/brtbrt123/sv-student-portal.git
cd sv-student-portal
```

```
npm install
npm run dev
```

The project needs a `.env.local` file with the Supabase URL and anon key (given by the Project Manager). The database scripts are in `docs/database`.

8. References

1. ACM, "ACM Code of Ethics and Professional Conduct," Association for Computing Machinery, 2018.
2. IEEE, "IEEE Code of Ethics," IEEE Policies, Section 7.8.
3. Supabase Inc., "Supabase Documentation: Database and Auth," 2026. [Online]. Available: https://supabase.com/docs
4. PostgreSQL Global Development Group, "PostgreSQL Documentation: Row Security Policies," 2026. [Online]. Available: https://www.postgresql.org/docs/
5. E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377-387, Jun. 1970.
6. K. Schwaber and J. Sutherland, "The Scrum Guide," Nov. 2020.
7. ISO/IEC/IEEE, "Systems and software engineering: Life cycle processes: Requirements engineering," ISO/IEC/IEEE 29148:2018, 2018.
8. Republic of the Philippines, "Data Privacy Act of 2012," Republic Act No. 10173, Aug. 2012.
9. Vercel Inc., "Next.js Documentation," 2026. [Online]. Available: https://nextjs.org/docs
10. OWASP Foundation, "OWASP Top 10: 2021," 2021. [Online]. Available: https://owasp.org/Top10/

9. Verification Reports

The reports below are attached as screenshots or exported files after the checks are run on the final document. Each result is recorded in the tables.

9.1 Plagiarism Checker Results

Item	Entry
Tool used	
Document checked	System Requirements & Project Documentation
Date checked	
Similarity result	%
Sources flagged and how they were handled	
Checked by	
Report attached	(insert screenshot or file name)

9.2 AI / Anti-AI Detector Results

Item	Entry
Tool used	
Document checked	System Requirements & Project Documentation
Date checked	
Result (percentage likely AI-generated / human-written)	
Notes on how the work was produced (see section 6.5, Use of AI assistance)	
Checked by	
Report attached	(insert screenshot or file name)

10. Team Monitoring Sheet

The Team Monitoring Sheet records the work of each member and the peer evaluation (team member grading). The full workbook (Team-Monitoring-Sheet.xlsx: task tracker, weekly progress, meeting log, peer evaluation and an automatic summary) is submitted with this document. The tables below are its printable summary.

10.1 Contribution summary

Member	Main tasks and deliverables	Tasks done	Hours	Remarks
Abarquez, Justinbril				
Cruzante, Aaron				
Canayong, Ron				
Esgana, Justine				
Espedilon, Arvien				
Dominado, Byrone				

10.2 Peer evaluation (team member grading)

Each member rates every teammate except themselves from 1 to 5 (5 = Excellent, 4 = Good, 3 = Satisfactory, 2 = Needs improvement, 1 = Poor). The table shows the average rating each member received.

Member evaluated	Contribution	Quality of work	Reliability	Communication	Teamwork	Initiative	Average
Abarquez, Justinbril							
Cruzante, Aaron							
Canayong, Ron							
Esgana, Justine							
Espedilon, Arvien							
Dominado, Byrone							

Evaluator signatures: _____